

第6讲 方程求解与最优化问题

(Chapter 6 Solution of Equations and Optimization)

6.1 线性方程组求解 (Solution of Linear Equations)

6.2 非线性方程数值求解 (Numerical Solution of Nonlinear Equations)

6.3 常微分方程的数值解法 (Numerical Solution of Ordinary Differential Equations)

6.4 最优化问题求解 (Optimization)

6.1 线性方程组求解 (Solution of Linear Equations)

6.1.1 直接解法 (Direct Solution)

1. 利用左除运算符的直接解法

对于线性方程组

$$Ax=b,$$

可以利用左除运算符 “\” 求解:

$$x=A\backslash b$$

Example 6-1 用直接解法求解下列线性方程组

$$\begin{cases} 2x_1 + x_2 - 5x_3 + x_4 = 13 \\ x_1 - 5x_2 + 7x_4 = -9 \\ 2x_2 + x_3 - x_4 = 6 \\ x_1 + 6x_2 - x_3 - 4x_4 = 0 \end{cases}$$

命令如下：

```
A=[2,1,-5,1;1,-5,0,7;0,2,1,-1;1,6,-1,-4];
```

```
b=[13,-9,6,0]';
```

```
x=A\b
```

2. 利用矩阵的分解求解线性方程组 (**Matrix Decomposition Method**)

矩阵分解是指根据一定的原理用某种算法将一个矩阵分解成若干个矩阵的乘积。

常见的矩阵分解有LU分解、QR分解、Cholesky分解，以及Schur分解、Hessenberg分解、奇异分解等。

(1) LU Decomposition

矩阵的LU分解就是将一个矩阵表示为一个下三角矩阵L和一个上三角矩阵U的乘积形式。（线性代数中已经证明，只要方阵A是非奇异的，LU分解总是可以进行的。）

MATLAB提供的lu函数用于对矩阵进行LU分解，其调用格式为：

[L,U]=lu(A)：产生一个上三角阵U和一个变换形式的下三角阵L(行交换)，使之满足 $A=LU$ 。注意，这里的矩阵A必须是方阵。

[L,U,P]=lu(A)：产生一个上三角阵U和一个下三角阵L以及一个置换矩阵P，使之满足 $PA=LU$ 。当然矩阵A同样必须是方阵。

实现LU分解后，线性方程组 $Ax=b$ 的解 $x=U\backslash(L\backslash b)$ 或 $x=U\backslash(L\backslash Pb)$ ，这样可以大大提高运算速度。

Example 6-2 用LU分解求解Example 6-1中的线性方程组。

命令如下：

```
A=[2,1,-5,1;1,-5,0,7;0,2,1,-1;1,6,-1,-4];
```

```
b=[13,-9,6,0]';
```

```
[L,U]=lu(A)
```

```
x=U\ (L\b)
```

或采用LU分解的第2种格式，命令如下：

```
[L,U,P]=lu(A);
```

```
x=U\ (L\ P*b)
```

(2) QR Decomposition

对矩阵A进行QR分解，就是把A分解为一个正交矩阵Q和一个上三角矩阵R的乘积形式。QR分解只能对方阵进行。MATLAB的函数`qr`可用于对矩阵进行QR分解，其调用格式为：

`[Q,R]=qr(A)`：产生一个正交矩阵Q和一个上三角矩阵R，使之满足 $A=QR$ 。

`[Q,R,E]=qr(A)`：产生一个正交矩阵Q、一个上三角矩阵R以及一个置换矩阵E，使之满足 $AE=QR$ 。

实现QR分解后，线性方程组 $Ax=b$ 的解 $x=R\backslash(Q\backslash b)$ 或 $x=E*(R\backslash(Q\backslash b))$ 。

Example 6-3 用QR分解求解Example 6-1中的线性方程组。

命令如下：

```
A=[2,1,-5,1;1,-5,0,7;0,2,1,-1;1,6,-1,-4];
```

```
b=[13,-9,6,0]';
```

```
[Q,R]=qr(A);
```

```
x=R\ (Q\b)
```

或采用QR分解的第2种格式，命令如下：

```
[Q,R,E]=qr(A);
```

```
x=E*(R\ (Q\b))
```


(3) Cholesky Decomposition

如果矩阵A是对称正定的，则Cholesky分解将矩阵A分解成一个下三角矩阵和上三角矩阵的乘积。设上三角矩阵为R，则下三角矩阵为其转置，即 $A=R'R$ 。MATLAB函数chol用于对矩阵A进行Cholesky分解，其调用格式为：

$R=\text{chol}(A)$ ：产生一个上三角阵R，使 $R'R=A$ 。若A为非对称正定，则输出一个出错信息。

$[R,p]=\text{chol}(A)$ ：这个命令格式将不输出出错信息。当A为对称正定的，则 $p=0$ ，R与上述格式得到的结果相同；否则p为一个正整数。如果A为满秩矩阵，则R为一个阶数为 $q=p-1$ 的上三角阵，且满足 $R'R=A(1:q,1:q)$ 。

实现Cholesky分解后，线性方程组 $Ax=b$ 变成 $R'Rx=b$ ，所以 $x=R \setminus (R' \setminus b)$ 。

Example 6-4 用Cholesky分解求解Example 6-1
中的线性方程组。

命令如下：

```
A=[2,1,-5,1;1,-5,0,7;0,2,1,-1;1,6,-1,-4];
```

```
b=[13,-9,6,0]';
```

```
R=chol(A)
```

```
??? Error using ==> chol
```

```
Matrix must be positive definite
```

命令执行时，出现错误信息，说明A为非正定矩阵。

6.1.2 迭代解法 (Iteration Solution)

迭代解法非常适合求解大型系数矩阵的方程组。在数值分析中，迭代解法主要包括 Jacobi迭代法、Gauss-Seidel迭代法、超松弛迭代法和两步迭代法。

迭代思想 (The idea of the iterative method)

为了求解线性方程组：

$$\begin{cases} 10x_1 - x_2 = 9 \\ -x_1 + 10x_2 - 2x_3 = 7 \\ -2x_2 + 10x_3 = 6 \end{cases}$$

将方程改写为：

$$\begin{cases} x_1 = 10x_2 - 2x_3 - 7 \\ x_2 = 10x_1 - 9 \\ x_3 = \frac{1}{10}(6 + 2x_2) \end{cases}$$

可以构造方程的迭代公式为：

$$\begin{cases} x_1^{(k+1)} = 10x_2^{(k)} - 2x_3^{(k)} - 7 \\ x_2^{(k+1)} = 10x_1^{(k)} - 9 \\ x_3^{(k+1)} = 0.6 + 0.2x_2^{(k)} \end{cases}$$

1. Jacobi Iteration

对于线性方程组 $\mathbf{Ax}=\mathbf{b}$ ，如果 $\mathbf{A}$ 为非奇异方阵，即 $a_{ii} \neq 0 (i=1,2,\dots,n)$ ，则可将 $\mathbf{A}$ 分解为 $\mathbf{A}=\mathbf{D}-\mathbf{L}-\mathbf{U}$ ，其中 $\mathbf{D}$ 为对角阵，其元素为 $\mathbf{A}$ 的对角元素， $\mathbf{L}$ 与 $\mathbf{U}$ 为 $\mathbf{A}$ 的下三角阵和上三角阵，于是 $\mathbf{Ax}=\mathbf{b}$ 化为：

$$\mathbf{x}=\mathbf{D}^{-1}(\mathbf{L}+\mathbf{U})\mathbf{x}+\mathbf{D}^{-1}\mathbf{b}$$

与之对应的迭代公式为：

$$\mathbf{x}^{(k+1)}=\mathbf{D}^{-1}(\mathbf{L}+\mathbf{U})\mathbf{x}^{(k)}+\mathbf{D}^{-1}\mathbf{b}$$

这就是Jacobi迭代公式。如果序列 $\{\mathbf{x}^{(k+1)}\}$ 收敛于 $\mathbf{x}$ ，则 $\mathbf{x}$ 必是方程 $\mathbf{Ax}=\mathbf{b}$ 的解。

Jacobi迭代法的MATLAB函数文件**Jacobi.m**如下:

```
function [y,n]=jacobi(A,b,x0,eps)
```

```
%%%x0为初始值
```

```
if nargin==3
```

```
    eps=1.0e-6;
```

```
elseif nargin<3
```

```
    error
```

```
    return
```

```
end
```

```
D=diag(diag(A)); %求A的对角矩阵
```

```
L=-tril(A,-1); %求A的下三角阵, 第-1条对角线以下的元素
```

```
U=-triu(A,1); %求A的上三角阵, 第1条对角线以上的元素
```

```
B=D\(L+U);
```

```
f=D\b;
```

```
y=B*x0+f;
```

```
n=1; %迭代次数
```

```
while norm(y-x0)>=eps
```

```
    x0=y;
```

```
    y=B*x0+f;
```

```
    n=n+1;
```

```
end
```

$$\mathbf{x}^{(k+1)} = \mathbf{D}^{-1}(\mathbf{L} + \mathbf{U})\mathbf{x}^{(k)} + \mathbf{D}^{-1}\mathbf{b}$$

Example 6-5 用Jacobi迭代法求解下列线性方程组。

$$\begin{cases} 10x_1 - x_2 = 9 \\ -x_1 + 10x_2 - 2x_3 = 7 \\ -2x_1 + 10x_2 = 6 \end{cases}$$

设迭代初值为0，迭代精度为 10^{-6} 。

在命令中调用函数文件Jacobi.m，命令如下：

```
A=[10,-1,0;-1,10,-2;0,-2,10];
```

```
b=[9,7,6]';
```

```
[x,n]=jacobi(A,b,[0,0,0]',1.0e-6)
```

$$\mathbf{D}\mathbf{x}^{(k+1)} = (\mathbf{L} + \mathbf{U})\mathbf{x}^{(k)} + \mathbf{b}$$

2. Gauss-Serdel Iteration

在Jacobi迭代过程中，计算 $\mathbf{x}_i^{(k+1)}$ 时， $\mathbf{x}_1^{(k+1)}, \dots, \mathbf{x}_{i-1}^{(k+1)}$ ，已经得到，不必再用 $\mathbf{x}_1^{(k)}, \dots, \mathbf{x}_{i-1}^{(k)}$ ，即原来的迭代公式 $\mathbf{D}\mathbf{x}^{(k+1)} = (\mathbf{L} + \mathbf{U})\mathbf{x}^{(k)} + \mathbf{b}$ 可以改进为 $\mathbf{D}\mathbf{x}^{(k+1)} = \mathbf{L}\mathbf{x}^{(k+1)} + \mathbf{U}\mathbf{x}^{(k)} + \mathbf{b}$ ，于是得到：

$$\mathbf{x}^{(k+1)} = (\mathbf{D} - \mathbf{L})^{-1}\mathbf{U}\mathbf{x}^{(k)} + (\mathbf{D} - \mathbf{L})^{-1}\mathbf{b}$$

该式即为Gauss-Serdel迭代公式。和Jacobi迭代相比，Gauss-Serdel迭代用新分量代替旧分量，精度会高些。

从第 k 次迭代到第 $k + 1$ 次迭代, x_i 的迭代公式为:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n$$

这里, $x_i^{(k)}$ 表示第 k 次迭代时 x_i 的值。

在计算 $x_i^{(k+1)}$ 时, 对于 $j < i$ 的 $x_j^{(k+1)}$ 已经在本次迭代中计算出来了,

所以使用这些最新的值; 而对于 $j > i$ 的 $x_j^{(k)}$ 则使用上一次迭代的值。

Gauss-Serdel迭代法的MATLAB函数文件**gauseidel.m**如下:

```
function [y,n]=gauseidel(A,b,x0,eps)
```

```
if nargin==3
```

```
    eps=1.0e-6;
```

```
elseif nargin<3
```

```
    error
```

```
    return
```

```
end
```

```
D=diag(diag(A)); %求A的对角矩阵
```

```
L=-tril(A,-1); %求A的下三角阵
```

```
U=-triu(A,1); %求A的上三角阵
```

```
G=(D-L)\U;
```

```
f=(D-L)\b;
```

```
y=G*x0+f;
```

```
n=1; %迭代次数
```

```
while norm(y-x0)>=eps
```

```
    x0=y;
```

```
    y=G*x0+f;
```

```
    n=n+1;
```

```
end
```

$$\mathbf{x}^{(k+1)} = (\mathbf{D} - \mathbf{L})^{-1} \mathbf{U} \mathbf{x}^{(k)} + (\mathbf{D} - \mathbf{L})^{-1} \mathbf{b}$$

Example 6-6 用Gauss-Serdel迭代法求解
Example 6-5中的线性方程组。

设迭代初值为0，迭代精度为 10^{-6} 。

在命令中调用函数文件gauseidel.m，命令如下：

```
A=[10,-1,0;-1,10,-2;0,-2,10];
```

```
b=[9,7,6]';
```

```
[x,n]=gauseidel(A,b,[0,0,0]',1.0e-6)
```

Example 6-7 分别用Jacobi迭代和Gauss-Serdel迭代法求解下列线性方程组，看是否收敛。

$$\begin{cases} x + 2y - 2z = 9 \\ x + y + z = 7 \\ 2x + 2y + z = 6 \end{cases}$$

命令如下：

```
a=[1,2,-2;1,1,1;2,2,1];
```

```
b=[9;7;6];
```

```
[x,n]=jacobi(a,b,[0;0;0])
```

```
[x,n]=gauseidel(a,b,[0;0;0])
```

Experiment Content

Ex-1: 分别用矩阵求逆、矩阵除法以及矩阵分解求下列线性方程组的解：

$$\begin{cases} 2x + 3y + 5z = 10 \\ 3x + 7y + 4z = 3 \\ x - 7y + z = 5 \end{cases}$$

Experiment Content

Ex-2: 下面是一个线性病态方程组：

$$\begin{pmatrix} 1/2 & 1/3 & 1/4 \\ 1/3 & 1/4 & 1/5 \\ 1/4 & 1/5 & 1/6 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 0.95 \\ 0.67 \\ 0.52 \end{pmatrix}$$

- (1) 求方程组的解；
- (2) 将方程右边向量元素 **b_3** 改为**0.53**，再求解，并比较 **b_3** 的变化和解的相对变化；
- (3) 计算系数矩阵 **A** 的条件数并分析结论。

6.2 非线性方程数值求解 (Numerical Solution of Nonlinear Equations)

6.2.1 单变量非线性方程求解 (Solving of nonlinear equations with one variable)

在MATLAB中提供了一个**fzero**函数，可以用来求单变量非线性方程的根。该函数的调用格式为：

$x = \text{fzero}('fname', x_0)$

$[x, fval, exitflag, output] = \text{fzero}(fun, x_0)$

其中**fname**是待求根的函数文件名，**x0**为搜索的起点，也可以是区间。一个函数可能有多个根，但**fzero**函数只给出离**x0**最近的那个根。

x：零点位置，**fval**：零点处的函数值，**exitflag**：1（成功找到零点），**output**：包含迭代次数，算法等信息的结构体。

Example 6-8 求 $f(x)=x-10^x+2=0$ 在 $x_0=0.5$ 附近的根。

步骤如下：

(1) 建立函数文件funx.m。

```
function fx=funx(x)
```

```
fx=x-10.^x+2;
```

(2) 调用fzero函数求根。

```
z=fzero('funx',0.5) %z=fzero(@funx,0.5)
```

```
z =
```

```
0.3758
```


6.2.2 非线性方程组的求解 (The solution of nonlinear equations)

对于非线性方程组 $F(X)=0$ ，用**fsolve**函数求其数值解。**fsolve**函数的调用格式为：

$X=fsolve('fun',X0,option) \%@fun$

其中 X 为返回的解，**fun**是用于定义需求解的非线性方程组的函数文件名， $X0$ 是求根过程的初值，**option**为最优化工具箱的选项设定。

最优化工具箱提供了20多个选项，用户可以使用 **optimset** 命令将它们显示出来。如果想改变其中某个选项，则可以调用 **optimset()** 函数来完成。

例如， 'Display' 选项决定函数调用时中间结果的显示方式，其中 'off ' 为不显示， 'iter' 表示每步都显示， 'final' 只显示最终结果。 **optimset('Display', 'off ')** 将设定 **Display** 选项为 'off '。

Example 6-9 求下列非线性方程组

$$\begin{cases} x - 0.6\sin x - 0.3\cos y = 0 \\ y - 0.6\cos x + 0.3\sin y = 0 \end{cases}$$

在(0.5,0.5) 附近的数值解。

(1) 建立函数文件myfun.m。

```
function q=myfun(p)
x=p(1);
y=p(2);
q(1)=x-0.6*sin(x)-0.3*cos(y);
q(2)=y-0.6*cos(x)+0.3*sin(y);
```

(2) 在给定的初值x0=0.5,y0=0.5下，调用fsolve函数求方程的根。

```
x=fsolve('myfun',[0.5,0.5],optimset('Display','off'))
```

```
x =  
0.6354  
0.3734
```

将求得的解代回原方程，可以检验结果是否正确，命令如下：

```
q=myfun(x)
```

```
q =  
1.0e-009 *  
0.2375 0.2957
```

可见得到了较高精度的结果。

6.3 常微分方程初值问题的数值解法 (Numerical method for initial value problems of ODE)

考虑常微分方程的初值问题

$$y' = f(t, y), t_0 \leq t \leq T, \quad y(t_0) = y_0$$

所谓其数值解法，就是求它的解 $y(t)$ 在节点 $t_0 < t_1 < \dots < t_m$ 处的近似值 $y_0, y_1, \dots, y_m$ 的方法。所求得的 $y_0, y_1, \dots, y_m$ 称为常微分方程初值问题的数值解。一般采用等距节点 $t_n = t_0 + nh$, $n=0, 1, \dots, m$, h 称为步长。

常用的有欧拉(Euler)法、龙格-库塔(Runge- Kutta)法、线性多步法、预报校正法等。

6.3.1 龙格—库塔法递推公式 (Runge-Kutta recurrence formula)

$$y(t_0 + ih) = y_i \approx y_{i-1} + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

其中

$$k_1 = f(t_{i-1}, y_{i-1})$$

$$k_2 = f(t_{i-1} + \frac{h}{2}, y_{i-1} + \frac{h}{2}k_1)$$

$$k_3 = f(t_{i-1} + \frac{h}{2}, y_{i-1} + \frac{h}{2}k_2)$$

$$k_4 = f(t_{i-1} + h, y_{i-1} + hk_3)$$

6.3.2 龙格—库塔法的实现 (Realization of the Runge-Kutta method)

基于龙格—库塔法，MATLAB提供了求常微分方程数值解的函数，一般调用格式为：

[t,y]=ode23('fname',tspan,y0)

[t,y]=ode45('fname',tspan,y0)

其中**fname**是定义**f(t,y)**的函数文件名，**该函数文件必须返回一个列向量**。**tspan**形式为**[t0,tf]**,表示求解区间。**y0**是初始状态列向量。**t**和**y**分别给出时间向量和相应的状态向量。

求常微分方程数值解的函数（the function of numerical solution of ODE）

求解函数	采用方法	适用场合
ode23	2-3 阶龙格—库塔算法，低精度	非刚性
ode45	4-5 阶龙格—库塔算法，中精度	非刚性
ode113	Adams 算法，精度可到 $10^{-3} \sim 10^{-6}$	非刚性，计算时间比 ode45 短
ode23t	梯形算法	适度刚性
ode15s	Gear's 反向数值微分算法，中精度	刚性
ode23s	2 阶 Rosebrock 算法，低精度	刚性，当精度较低时，计算时间比 ode15s 短
ode23tb	梯形算法，低精度	刚性，当精度较低时，计算时间比 ode15s 短

若微分方程描述的是一个变化过程包含着多个相互作用但变化速度相差十分悬殊的子过程，这样一类过程就认为具有“刚性”，求解刚性方程的初值问题的解析解是困难的，常采用 ode15s、ode23s和 ode23tb求其数值解。求解非刚性的一阶常微分方程或方程组的初值问题的数值解常采用函数ode23和 ode45。

Example 6-10 设有初值问题

$$y' = \frac{y^2 - t - 2}{4(t+1)}, 0 \leq t \leq 10, y(0) = 2$$

求其数值解，并与精确解比较(精确解为 $y = \sqrt{t+1} + 1$)。

(1) 建立函数文件funt.m。

```
function yp=funt(t,y)
```

```
yp=(y^2-t-2)/4/(t+1);
```

(2) 求解微分方程。

```
t0=0;tf=10;
```

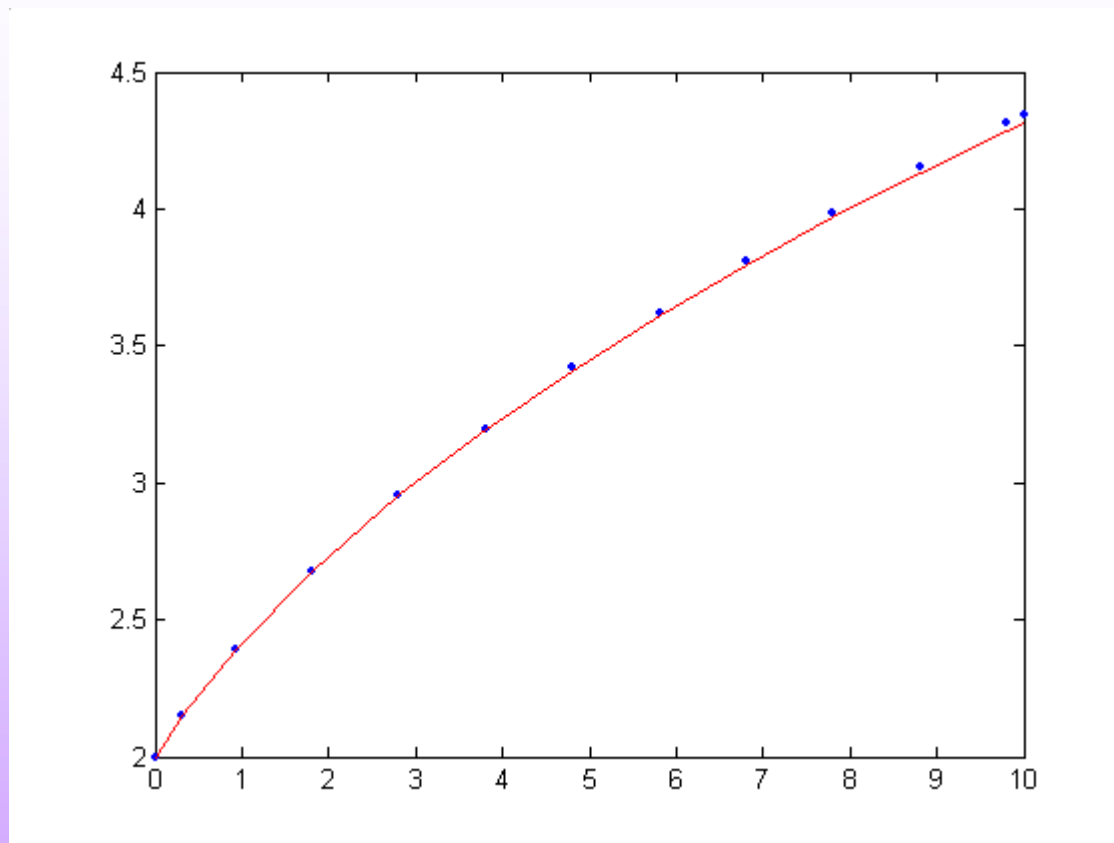
```
y0=2;
```

```
[t,y]=ode23('funt',[t0,tf],y0); %求数值解
```

```
y1=sqrt(t+1)+1; %求精确解
```

```
plot(t,y, 'b.',t,y1, 'r-'); %通过图形来比较
```

结果如下



数值解为蓝色圆点，精确值为红色实线，显然两者结果近似。

Example 6-11 已知一个二阶线性系统微分方程为

$$\begin{cases} \frac{d^2 x}{dt^2} + ax = 0, & a > 0 \\ \underline{x}(0) = 0, & \underline{x}'(0) = 1 \end{cases}$$

其中 $a=2$ ，绘制系统的时间响应曲线和相平面图。

分析：令 $x_2=x$ ， $x_1=x'$ ，则系统的状态方程：

$$\begin{cases} x_2' = x_1 \\ x_1' = -ax_2 \\ x_2(0) = 0, x_1(0) = 1 \end{cases}$$

建立函数文件：

```
function xdot=sys(t,x)
```

```
xdot=[-2*x(2);x(1)];
```

取 $t_0=0$, $t_f=20$, 求微分方程的解:

```
t0=0;tf=20;
```

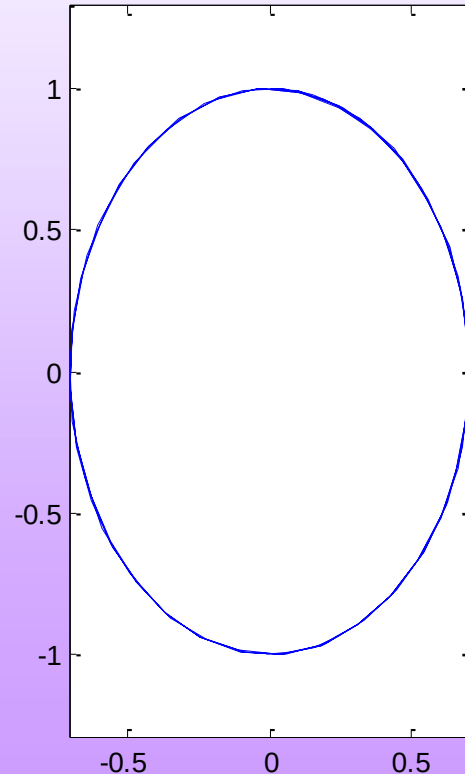
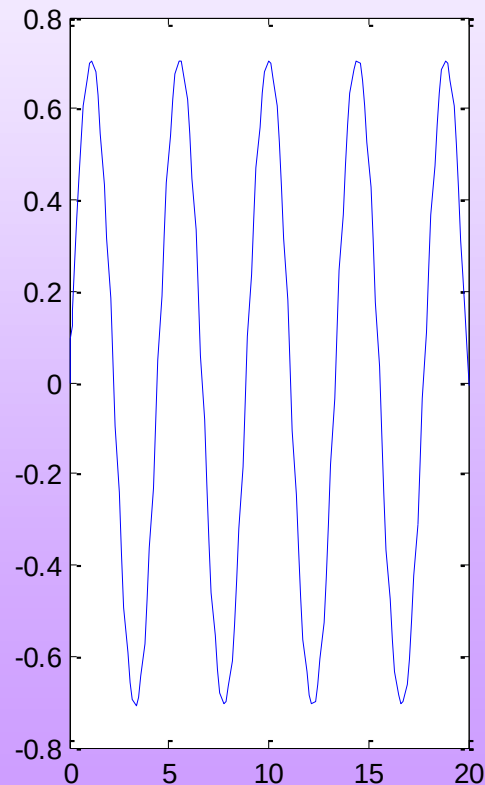
```
[t,x]=ode45('sys',[t0,tf],[1,0]) ;
```

```
[t,x]
```

```
ans =      t          x'          x  
      0      1.0000          0  
 0.0001      1.0000  0.0001  
.....  
19.9170 -0.9937  0.0738  
20.0000 -0.9991 -0.0090
```

$$\begin{cases} x_2' = x_1 \\ x_1' = -ax_2 \\ x_2(0) = 0, x_1(0) = 1 \end{cases}$$

```
subplot(1,2,1);plot(t,x(:,2)); %解的曲线t-x  
subplot(1,2,2);plot(x(:,2),x(:,1)); %相平面曲线x-x'  
axis equal
```



6.4 最优化问题求解 (Optimization Problem Solving)

6.4.1 无约束最优化问题求解 (Unconstrained Optimization)

MATLAB提供了3个最小值函数，它们的调用格式为：**`[x,fval]=fminbnd('fname',x1,x2)`**

`[x,fval]=fminsearch('fname',x1,x2)`

`[x,fval]=fminunc('fname',x0)`

其中fminbnd用于求一元函数在区间(x1,x2)中的最小值点x和最小值fval。fminsearch函数是基于单纯形算法，fminunc是基于拟牛顿法求多元函数的最小值点，x0是求解的初始值向量。

注意：

1. 当目标函数的阶数大于2时，使用**fminunc**比**fminsearch**更有效。但当目标函数高度不连续时，使用**fminsearch**效果更好
2. **MATLAB**没有专门提供求函数最大值的函数，但只要注意到 $-f(x)$ 在区间 (a,b) 上的最小值就是 $f(x)$ 在 (a,b) 的最大值，所以

fmin(-f,x1,x2)

返回函数 $f(x)$ 在区间 $(x1,x2)$ 上的最大值点。

Example 6-12 求 $f(x)=x^3-2x-5$ 在 $[0,5]$ 内的最小值点与最大值点。

(1) 建立函数文件mymin.m。

```
function fx=mymin(x)
```

```
fx=x.^3-2*x-5;
```

(2) 调用fmin函数求最小值点与最大值点。

```
xmin=fminbnd('mymin',0,5)%@(x) x.^3-2*x-5
```

```
xmin=
```

```
0.8165
```

```
xmax= fminbnd(@(x) -(x.^3-2*x-5),0,5)
```


Example 6-13 求函数 $f(x)=x+y^2/(4x)+z^2/y+2/z$ 在 $(0.5,0.5,0.5)$ 附近的最小值

(1) 建立函数文件fxy.m。

```
function f=fxy(p)
```

```
x=p(1);y=p(2);z=p(3);f=x+y^2/4/x+z^2/y+2/z;
```

(2) 调用fminsearch函数求最小值点。

```
[u,fmin]=fminsearch('fxy',[0.5,0.5,0.5])
```

```
u=
```

```
0.5000 1.0000 1.0000
```

```
fmin=
```

```
4.0000
```

6.4.2 有约束最优化问题求解 (Constrained Optimization)

问题描述: $\min f(x)$

$$\text{s.t. } G(x) \leq 0$$

约束条件可细化为:

- (1) 线性不等式约束: $Ax \leq b$
- (2) 线性等式约束: $A_{eq}x = b_{eq}$
- (3) 非线性不等式约束: $Cx \leq 0$
- (4) 非线性等式约束: $C_{eq}x = 0$
- (5) x 的下界和上界: $L_{bnd} \leq x \leq U_{bnd}$

fmincon函数的调用格式:

[x,fval]=fmincon(@fname,x0,A,b,Aeq,beq,Lbnd,Ubnd,NonF,options)

参数**NonF**为非线性约束函数的M文件名。如果某个约束不存在, 则用空矩阵来表示。

Example 6-14 $\min f(X) = -2x_1 - x_2$
s.t. $g_1(X) = 25 - x_1^2 - x_2^2 \geq 0$
 $g_2(X) = 7 - x_1^2 + x_2^2 \geq 0$
 $0 \leq x_1 \leq 5, 0 \leq x_2 \leq 10$

1. 先建立M-文件**fun.m**定义目标函数:

```
function f=fun(x);  
f=-2*x(1)-x(2);
```

2. 再建立M文件**mycon2.m**定义非线性约束:

```
function [g,ceq]=mycon2(x)  
g=[x(1)^2+x(2)^2-25;x(1)^2-x(2)^2-7];  
ceq=[];
```

命令如下：

```
x0=[3;2.5];
```

```
Lbnd=[0 0];
```

```
Ubnd=[5 10];
```

```
[x, fval]=fmincon('fun',x0,[],[],[],[],Lbnd,Ubnd,'mycon2')
```

```
x =
```

```
    4.0000
```

```
    3.0000
```

```
fval =
```

```
   -11.0000
```

6.4.3 线性规划问题求解 (Linear programming problem solving)

线性规划问题的标准形似：

$$\text{Min } f(x)$$

$$\text{s.t. } Ax \leq b$$

$$A_{eq}x = b_{eq}$$

$$L_{bnd} \leq x \leq U_{bnd}$$

linprog函数的调用格式：

$$[x, fval] = \text{linprog}(f, A, b, Aeq, beq, lbnd, ubnd)$$

其中，**f**为目标函数的系数向量。若没有等式约束，则令**Aeq**=[], **beq**=[]。

Example 6-15

$$\begin{aligned} \max \quad & z = 0.4x_1 + 0.28x_2 + 0.32x_3 + 0.72x_4 + 0.64x_5 + 0.6x_6 \\ \text{s.t.} \quad & 0.01x_1 + 0.01x_2 + 0.01x_3 + 0.03x_4 + 0.03x_5 + 0.03x_6 \leq 850 \\ & 0.02x_1 + 0.05x_4 \leq 700 \\ & 0.02x_2 + 0.05x_5 \leq 100 \\ & 0.03x_3 + 0.08x_6 \leq 900 \\ & x_j \geq 0 \quad j = 1, 2, \dots, 6 \end{aligned}$$

f=[-0.4 -0.28 -0.32 -0.72 -0.64 -0.6];

A=[0.01 0.01 0.01 0.03 0.03 0.03;0.02 0 0 0.05 0 0;0 0.02 0 0 0.05 0;0 0 0.03 0 0 0.08];

b=[850;700;100;900];

Aeq=[]; beq=[];

vlb=[0;0;0;0;0;0]; vub=[];

[x,fval]=linprog(f,A,b,Aeq,beq,vlb,vub)

x =
1.0e+04 *
3.5000
0.5000
3.0000
0.0000
0.0000
0.0000
fval =
-2.5000e+04

Example 6-16

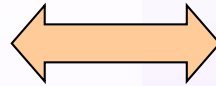
$$\min z = 6x_1 + 3x_2 + 4x_3$$

$$s.t. \quad x_1 + x_2 + x_3 = 120$$

$$x_1 \geq 30$$

$$0 \leq x_2 \leq 50$$

$$x_3 \geq 20$$



$$\begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} \leq \begin{pmatrix} 120 \\ 50 \end{pmatrix}$$

$$\begin{pmatrix} 30 \\ 0 \\ 20 \end{pmatrix} \leq \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$$

f=[6 3 4];

A=[0 1 0];

b=[50];

Aeq=[1 1 1];

beq=[120];

vlb=[30,0,20];

vub=[];

[x,fval]=linprog(f,A,b,Aeq,beq,vlb,vub)

x =

30.0000

50.0000

40.0000

fval =

490.0000

Experiment Content

Ex-1: Lorenz模型的状态方程表示为:

$$\begin{cases} x'_1(t) = -ax_1(t) + x_2(t)x_3(t) \\ x'_2(t) = -bx_2(t) + bx_3(t) \\ x'_3(t) = -x_2(t)x_1(t) + cx_2(t) - x_3(t) \end{cases}$$

取 $a=8/3$, $b=10$, $c=28$, 试绘制系统相平面图。

Ex-2: 求常微分方程的数值解。

$$(1+t^2)y'' + 2ty' + 3y = 2, \quad t_0=0, \quad t_f=5, \quad y(t_0)=0, \quad y'(t_0)=1$$

Experiment Content

Ex-3: 求下列方程（组）在初始值附近的根。

(1) $x^{41} + x^3 + 1 = 0$, $x_0 = -1$;

(2) $x - (1/x) \sin x = 0$, $x_0 = 0.5$.

(3)
$$\begin{cases} x^2 + y^2 = 9 \\ x + y = 1 \end{cases}, \quad x_0 = 3, \quad y_0 = 0.$$

Experiment Content

Ex-4: 求下列优化问题的解。

(1)

$$\min f(x) = 0.4x_2 + x_1^2 + x_2^2 - x_1x_2 + x_1^3$$

$$\text{s.t. } x_1 + 0.5x_2 \geq 0.4$$

$$0.5x_1 + x_2 \geq 0.5$$

$$x_1 \geq 0, x_2 \geq 0$$

(2)

$$\min S = xy + 2xz + 2yz$$

$$\text{s.t. } xyz = 10$$

$$x, y, z > 0$$